📡 3GPP 6G THz Network Simulator & Live Dashboard

A self-contained, real-time physics engine and data visualization dashboard built entirely in Python and Matplotlib. This tool simulates 6G Terahertz (THz) network behaviors, applying physical channel models to dynamic node mobility to generate live Key Performance Indicators (KPIs) like SINR, Throughput, and Latency.

🚀 Overview

This simulator provides a macroscopic, visual-first approximation of 6G THz environments (100 GHz – 1 THz). Rather than relying on heavy, non-visual packet-level simulators, this script calculates live RF physics and renders them instantly across a 12-panel interactive Matplotlib dashboard.

Key Visualizations Included:

- **Per-Node Spider/Radar Charts:** 6-axis KPI wheel tracking SINR, Throughput, Latency, Coverage, Beamforming Gain, and Reliability.
- **Dynamic Heatmaps:** Live-refreshing plasma and contour overlays representing the RF field.
- **Polar SINR Maps:** Angular SINR distribution radiating from the primary base station.
- **Per-Service Isolation Maps:** Side-by-side heatmaps filtered by network slice (URLLC, eMBB, mMTC, XR, V2X).
- **Rolling SINR Distributions:** Live histograms comparing signal quality against historical means and medians.
- **Live Timelines:** Time-series tracking of systemic handovers, aggregated throughput, and individual node latencies.

🧮 Physics & Mathematical Models

The engine utilizes NS3-aligned mathematical models to approximate THz channel characteristics:

- **Path Loss:** Friis Free-Space Path Loss combined with THz-specific Molecular Absorption (simulating oxygen and water vapor attenuation).

- **Massive MIMO:** Applies raw beamforming gain (18–40 dBi) to overcome the extreme propagation losses inherent to the THz band.
- **Capacity:** Approximates maximum theoretical data rates using the Shannon-Hartley theorem (with an efficiency factor of $\eta = 0.65$ to represent protocol overhead).
- **Environmental Fading:** Implements shadowing ($\sigma = 1.8$ dB) and static penetration losses for buildings, walls, and obstacles.

Pre-Configured Use Cases

The simulator ships with 6 distinct environments, each tuned with specific RF parameters, node mobility patterns, and latency targets:

1. **XR Surgery:** Holographic remote surgery · 300 GHz · <1 ms URLLC
2. **Auto Factory:** Industry 6.0 smart factory · 140 GHz · URLLC + mMTC
3. **Smart Intersection:** 6G V2X crossroad · 300 GHz · Zero-accident <0.5 ms latency
4. **THz Backhaul:** 1 THz rooftop P2P backhaul · 300 GHz Bandwidth · 1 Tbps targets
5. **Tunnel Rescue:** 6G rescue in confined spaces · 100 GHz · Critical URLLC
6. **Holo Classroom:** Holographic tele-education · 300 GHz · 10 Gbps/user eMBB

Installation & Requirements

This script is designed to be highly portable with minimal dependencies.

Prerequisites

- Python 3.8+
- numpy
- matplotlib
- scipy

Interactive Controls

The dashboard features on-screen clickable UI widgets, as well as several keyboard shortcuts for live manipulation:

- `SPACE` : Pause / Resume simulation
- `R` : Reset current environment
- `E` : Export current node state and historical metrics to a `.csv` file
- `1 - 6` : Instantly switch between the 6 pre-configured environments
- `+ / -` : Increase / Decrease simulation speed
- `H` : Toggle primary SINR Heatmap on/off (saves CPU)

- T : Toggle node mobility Trails on/off
- L : Toggle active gNB Link lines on/off

🏗 Code Architecture

- **ENVIRONMENTS Dictionary:** The master configuration block. Defines hardware specs, frequencies, map layouts, base stations, and moving nodes.
- **SimState Class:** The brain of the simulator. Tracks node kinematics (position, velocity, bounding box collisions), triggers physics calculations, tracks handovers, and manages historical data queues.
- **step() Function:** Advances the simulation by a fixed time delta (dt=0.04s), updating positions and recalculating RF physics.
- **draw_frame() Loop:** Powered by FuncAnimation, this routine clears and redraws all 12 Matplotlib axes at 60 FPS using the fresh data from SimState.

📄 Data Export

Pressing the E key (or clicking the "Export CSV" button) will instantly dump the current simulation state to a CSV file in your working directory (e.g., thz_sinr_XR_Surgery_1700000000.csv). The export includes:

- Node ID, Type, and Service Slice
- X/Y Coordinates
- Current SINR (dB)
- Current Throughput (Gbps)
- Current Latency (ms)

1. High-Level Architecture

The script is divided into several main components working together in a loop:

- **The Environments (ENVIRONMENTS):** This dictionary acts as the configuration file. It defines six distinct scenarios (like "Smart Intersection" for V2X or "XR Surgery"). It hardcodes the RF parameters, building boundaries, base station (gNB) locations, and moving nodes (cars, drones, sensors).
- **The Physics Engine:** Functions like thz_abs_db, friis_db, and compute_sinr handle the real-world mathematical modeling of radio waves in the 100 GHz – 1 THz range.
- **The Simulation State (SimState):** This is the "brain." Every time the animation loop ticks, the step() function calculates where every node has moved, determines their new distances to the gNBs, and recalculates their network metrics.

- **The Visualizer (draw_frame):** Powered by matplotlib.animation.FuncAnimation, this massive function clears the old charts and redraws the heatmaps, scatter plots, and radar charts using the fresh data from SimState.

2. The Simulation Parameters

To model the THz channel realistically, the simulation relies on a specific set of RF parameters defined inside the ENVIRONMENTS dictionary.

Here are the core parameters driving the physics:

- **freq_hz (Carrier Frequency):** Ranges from 100×10^9 Hz (100 GHz) up to 1.0×10^{12} Hz (1 THz). Higher frequencies provide more bandwidth but suffer from severe molecular absorption.
- **bw_ghz (Bandwidth):** Massive channel bandwidths characteristic of 6G, ranging from 30 GHz to 300 GHz. This is the primary driver of the enormous throughput numbers.
- **tx_power_dbm (Transmit Power):** The baseline signal strength leaving the base station, ranging from 20 to 45 dBm.
- **beamforming_gain_db (Massive MIMO Gain):** Extremely directional antennas are required in THz communications to overcome the massive path loss. This parameter (18 - 40 dB) acts as a raw bonus to the received signal strength.
- **pen_loss_avg (Penetration Loss):** A static penalty applied to simulate signal degradation when passing through obstacles (walls, glass, concrete).
- **noise_floor_dbm (Thermal Noise):** The background electromagnetic noise (usually around -80 dBm). Signals must stay sufficiently above this line to be decoded.

3. How the Values are Generated (The Physics Engine)

Every 40 milliseconds (defined by dt=0.04 in the step function), the code generates new values for Signal-to-Interference-plus-Noise Ratio (SINR), Throughput, and Latency for every node. Here is the mathematical pipeline:

Step A: Calculating Path Loss

First, the code calculates how much the signal degrades over the physical distance between the base station and the node. It uses two functions for this:

1. **Free-Space Path Loss (friis_db):** Calculates how the signal spreads out and weakens over distance in a vacuum.

$$PL_{Friis} = 20 \log_{10} \left(\frac{4 \pi d f}{c} \right)$$

(Where d is distance, f is frequency, and c is the speed of light).

2. **Molecular Absorption (`thz_abs_db`):** THz waves are notoriously absorbed by water vapor and oxygen molecules. The script uses a simplified lookup table `_THZ_ABS` to apply an attenuation factor (k) based on the specific frequency.

Step B: Computing SINR (`compute_sinr`)

Once the total loss is known, the engine calculates the Received Power (P_{rx}) for every base station in the area:

$$P_{rx} = P_{tx} - PL_{Total} + G_{Beamforming} + \text{Shadowing}$$
(Shadowing is generated using a random normal distribution `rng.normal(0, 1.8)` to simulate random environmental scattering).

The base station providing the strongest P_{rx} is locked onto as the "Signal" (S). The power from all other base stations is summed up to act as "Interference" (I). The final SINR is calculated as:

$$SINR_{dB} = 10 \log_{10} \left(\frac{S}{I + N} \right)$$
Step C: Deriving Throughput and Latency

With the SINR established, the code translates that signal quality into user-facing metrics.

1. **Throughput (`shannon_tp`):** It calculates the maximum theoretical data rate using a modified Shannon-Hartley theorem, applying an efficiency factor ($\eta = 0.65$) to account for real-world protocol overhead:

$$Capacity = B \log_2(1 + 10^{\frac{SINR}{10}}) \times 0.65$$

2. **Latency:** This is approximated dynamically based on signal strength. As SINR drops, the code assumes more packet retransmissions are necessary, exponentially increasing the delay:

$$Latency = Latency_{Target} \times 2 e^{-\frac{SINR}{15}}$$
By running these equations on every node, every frame, the arrays (`sinr_hist`, `tp_hist`, `lat_hist`) fill up with data, which Matplotlib then pulls from to draw the live dashboards.